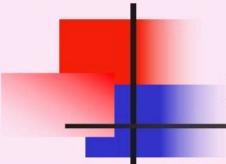


数据库系统概论

An Introduction to Database System

第十二章 并发控制





并发控制概述

多事务执行方式

(1)事务串行执行

(2)事务的交错执行

(3)事务的并行执行

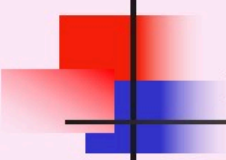


数据不一致实例：飞机订票系统

事务 T ₁	事务 T ₂
① 读A=16	
②	读A=16
③ A←A-1 写回A=15	
④	A←A-3 写回A=13

T₁的修改被T₂覆盖了！





并发操作带来的数据不一致性

- 丢失修改 (lost update)
- 不可重复读 (non-repeatable read)
- 读“脏”数据 (dirty read)





1. 丢失修改

丢失修改是指事务1与事务2从数据库中读入同一数据并修改
事务2的提交结果破坏了事务1提交的结果，
导致事务1的修改被丢失。



图12.1 三种数据不一致性

T ₁	T ₂
① 读A=16	读A=16
②	
③ A←A-1 写回 A=15	A←A-3 写回A=13
④	

(a) 丢失修改





2. 不可重复读

不可重复读是指事务1读取数据后，事务2执行更新操作，使事务1无法再现前一次读取结果。



图12.1 三种数据不一致性(续)

T ₁	T ₂
① 读A=50 读B=100 求和=150	
②	读B=100 B ← B*2 写回B=200
③ 读A=50 读B=200 求和=250 (验算不对)	

(b) 不可重复读



1. 事务2对其做了修改，当事务1再次读该数据时，得到与前一次不同的值。

2. 事务2删除了其中部分记录，当事务1再次读取数据时，发现某些记录神秘地消失了。

3. **事务2**插入了一些记录，当事务1再次按相同条件读取数据时，发现多了一些记录。

后两种不可重复读有时也称为**幻影现象**（phantom row）



3. 读“脏”数据

事务1修改某一数据，并将其写回磁盘

事务2读取同一数据后

事务1由于某种原因被撤消，这时事务1已修改过的数据恢复原值

事务2读到的数据就与数据库中的数据不一致，是不正确的数据，又称为“脏”数据。



图12.1 三种数据不一致性(续)

T ₁	T ₂
① 读C=100 C←C*2 写回C	
②	读C=200
③ ROLLBACK C恢复为100	

(c) 读“脏”数据





一、什么是封锁

- 封锁就是事务T在对某个数据对象（例如表、记录等）操作之前，先向系统发出请求，对其加锁。
- 加锁后事务T就对该数据对象有了一定的控制，在事务T释放它的锁之前，其它的事务不能更新此数据对象。
- 封锁是实现并发控制的一个非常重要的技术





二、基本封锁类型

- DBMS通常提供了多种类型的封锁。
- 一个事务对某个数据对象加锁后究竟拥有什么样的控制是由封锁的类型决定的。
- 基本封锁类型
 - 排他锁（Exclusive lock，简记为X锁）
 - 共享锁（Share lock，简记为S锁）



排他锁

- 排他锁又称为写锁
- 若事务T对数据对象A加上X锁，则只允许T读取和修改A，其它任何事务都不能再对A加任何类型的锁，直到T释放A上的锁





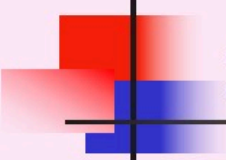
共享锁

- 共享锁又称为读锁
- 若事务T对数据对象A加上S锁，则其它事务只能再对A加S锁，而不能加X锁，直到T释放A上的S锁



实例

T ₁	T ₂
① <u>Xlock A</u> 获得	
② <u>读A=16</u>	
③ <u>A←A-1</u> 写回A=15 <u>Commit</u> <u>Unlock A</u>	<u>Xlock A</u> 等待 等待 等待 等待
④	<u>获得Xlock A</u> <u>读A=15</u> A←A-1 写回A=14 <u>Commit</u> <u>Unlock A</u>
⑤	



12.5 活锁和死锁

- 封锁方法

- 活锁

- 死锁



(1) 活锁

T ₁	T ₂	T ₃	T ₄
<u>lock R</u>	.	.	.
.	<u>lock R</u>	.	.
.	等待	<u>Lock R</u>	.
<u>Unlock</u>	等待	.	<u>Lock R</u>
.	等待	<u>Lock R</u>	等待
.	等待	.	等待
.	等待	<u>Unlock</u>	等待
.	等待	.	Lock R
.	等待	.	.

↓
t



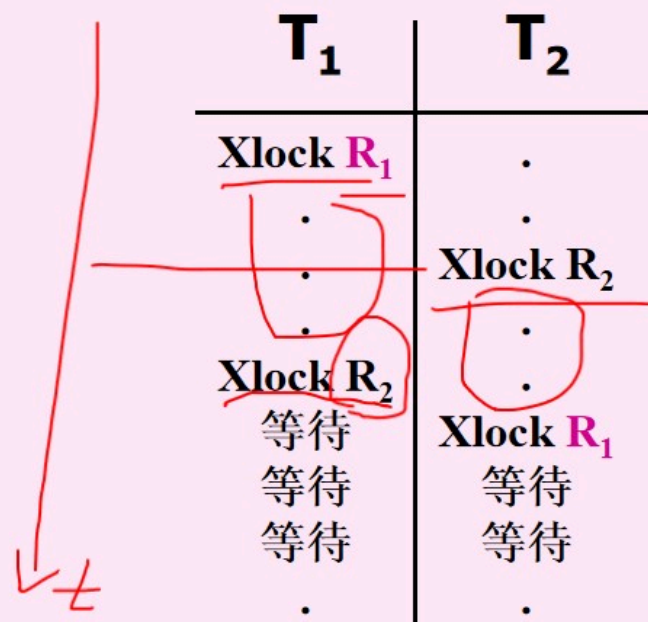
如何避免活锁

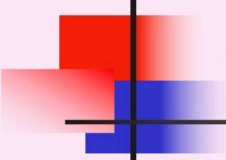
采用先来先服务的策略：

当多个事务请求封锁同一数据对象时

- 按请求封锁的先后次序对这些事务排队
- 该数据对象上的锁一旦释放，首先批准申请队列中第一个事务获得锁。

(2) 死锁





解决死锁的方法

两类方法

1. 预防死锁
2. 死锁的诊断与解除





死锁的预防（续）

- 在操作系统中广为采用的预防死锁的策略并不很适合数据库的特点
- **DBMS**在解决死锁的问题上更普遍采用的是诊断并解除死锁的方法





2. 死锁的诊断与解除

- 允许死锁发生
- 解除死锁
 - 由DBMS的并发控制子系统定期检测系统中是否存在死锁
 - 一旦检测到死锁，就要设法解除



检测死锁：超时法

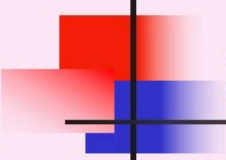
- 如果一个事务的等待时间超过了规定的时限，就认为发生了死锁
- 优点：实现简单
- 缺点
 - 有可能误判死锁
 - 时限若设置得太长，死锁发生后不能及时发现



并发调度的可串行性

- 几个事务的并行执行是正确的，当且仅当其结果与按某一次序串行地执行它们时的结果相同。这种并行调度策略称为可串行化（**Serializable**）的调度。





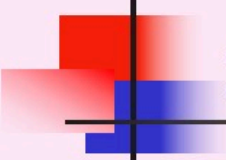
12.8 封锁的粒度

1 封锁粒度

2 多粒度封锁

3 意向锁





12.8.1 封锁粒度

一、什么是封锁粒度

二、选择封锁粒度的原则





一、什么是封锁粒度

- X锁和S锁都是加在某一个数据对象上的
- 封锁的对象:逻辑单元, 物理单元
- 例: 在关系数据库中, 封锁对象:
 - 逻辑单元: 属性值、属性值集合、元组、关系、索引项、整个索引、整个数据库等
 - 物理单元: 页(数据页或索引页)、物理记录等



什么是封锁粒度（续）

- 封锁对象可以很大也可以很小
例：对整个数据库加锁
对某个属性值加锁
- 封锁对象的大小称为封锁的粒度(Granularity)
- 多粒度封锁(multiple granularity locking)
 - 在一个系统中同时支持多种封锁粒度供不同的事务选择





二、选择封锁粒度的原则

- 封锁的粒度 大, 小,
 - 系统被封锁的对象 少, 多,
 - 并发度 低, 高,
 - 系统开销 小, 大,
-
- 选择封锁粒度:
 - 考虑封锁并发度因素
 - 对系统开销与并发度进行权衡



选择封锁粒度的原则（续）

- 需要处理多个关系的大量元组的用户事务：以数据库为封锁单位；
- 需要处理大量元组的用户事务：以关系为封锁单元；
- 只处理少量元组的用户事务：以元组为封锁单位





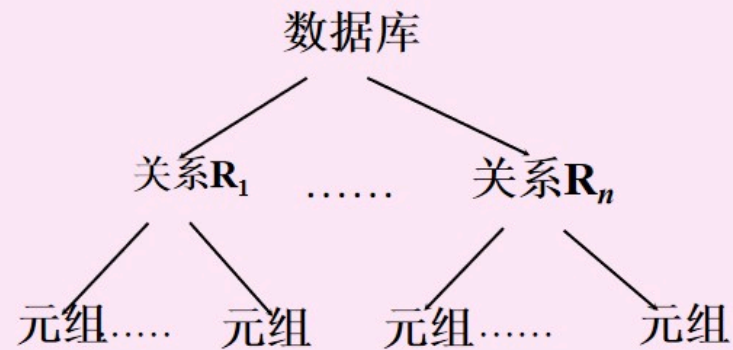
12.8.2 多粒度封锁

- 多粒度树
- 以树形结构来表示多级封锁粒度
 - 根结点是整个数据库，表示最大的数据粒度
 - 叶结点表示最小的数据粒度





例：三级粒度树。根结点为数据库，数据库的子结点为关系，关系的子结点为元组。





多粒度封锁协议

- 允许多粒度树中的每个结点被独立地加锁
- 对一个结点加锁意味着这个结点的所有后继结点也被加以同样类型的锁
- 在多粒度封锁中一个数据对象可能以两种方式封锁：**显式封锁和隐式封锁**





显式封锁和隐式封锁

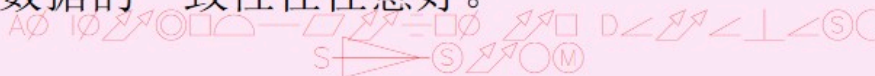
- 显式封锁：直接加到数据对象上的封锁
- 隐式封锁：由于其上级结点加锁而使该数据对象加上了锁
- 显式封锁和隐式封锁的效果是一样的





小结

- 数据共享与数据一致性是一对矛盾
- 数据库的价值在很大程度上取决于它所能提供的数据共享度。
- 数据共享在很大程度上取决于系统允许对数据并发操作的程度。
- 数据并发程度又取决于数据库中的并发控制机制
- 另一方面，数据的一致性也取决于并发控制的程度。施加的并发控制愈多，数据的一致性往往愈好。





小结（续）

- 活锁：先来先服务
- 死锁：
 - 预防方法
 - 一次封锁法
 - 顺序封锁法
 - 死锁的诊断与解除
 - 超时法
 - 等待图法





小结（续）

- 不同的数据库管理系统提供的封锁类型、封锁协议、达到的系统一致性级别不尽相同。
- 但是其依据的基本原理和技术是相同的。

